



**UNIVERSIDAD
DEL AZUAY**

Facultad de Ciencias de la Administración

Escuela de Ciencias de la Computación

Ingeniería de Software II

Tema: Módulo C — Cuentas por Pagar
Sistema Integrado de Gestión Empresarial (SIGE)

Autores: Andrés Merchán, Daniel Maldonado, Marco Rojas, Ariel Serrano

Profesor: PhD. Patricia Ortega

Fecha de elaboración: 28 de Abril de 2026

Índice

a) Introducción	3
b) REQUERIMIENTOS FUNCIONALES DEL MÓDULO.....	4
c) Modelo funcional: Diagrama de casos de uso.....	8
d) Modelo estructural: Diagrama de clases	19
e) Modelo comportamental: Diagramas de secuencia.....	38
f) Diseño arquitectónico: Diagrama de componentes	41
g) Aplicación de patrones de diseño.....	44
h) Matriz de trazabilidad	46
i) Conclusiones individuales	49
Declaración de uso de herramientas de IA	51
j) Referencias bibliográficas.....	52

a) Introducción

Descripción del módulo

El Módulo C — Cuentas por Pagar es el subsistema del Sistema Integrado de Gestión Empresarial (SIGE) responsable de gestionar el ciclo completo de compras de una pequeña o mediana empresa comercial. Su función principal es administrar las relaciones financieras de la empresa con sus proveedores: desde la emisión de una orden de compra hasta la cancelación total de las obligaciones generadas por las adquisiciones realizadas.

Este módulo cubre las siguientes funciones principales: el registro y mantenimiento del catálogo de proveedores, la creación y seguimiento de órdenes de compra, el registro de recepciones de mercadería, el ingreso de facturas emitidas por proveedores, la programación y ejecución de pagos, y la generación de reportes de obligaciones pendientes y estados de cuenta por proveedor.

Desde la perspectiva del diseño orientado a objetos, el módulo se estructura en torno a entidades claramente identificables del dominio empresarial — Proveedor, OrdenCompra, Recepcion, FacturaProveedor, CuentaPorPagar, PagoProveedor — con responsabilidades bien definidas y relaciones que reflejan el flujo real del proceso de compra.

Alcance funcional del módulo: registro y mantenimiento de proveedores, creación y seguimiento de órdenes de compra, registro de recepciones de mercadería, ingreso de facturas de compra, generación de cuentas por pagar, programación y registro de pagos, conciliación de saldos y generación de reportes de obligaciones pendientes.

Objetivos del módulo

- Gestionar el ciclo completo de compra desde la orden hasta el pago, garantizando trazabilidad en cada etapa.
- Integrarse con el Módulo B (Inventarios) para actualizar automáticamente el stock al recibir mercadería.
- Proveer información actualizada sobre las obligaciones financieras pendientes de la empresa con sus proveedores.
- Generar alertas sobre pagos próximos a vencer para facilitar la gestión de la tesorería.
- Integrarse con el Módulo D (Gestión de Impuestos) para el cálculo y registro de retenciones en la fuente y del IVA pagado en cada factura de compra registrada.

Alcance del trabajo

El presente informe documenta el análisis, diseño y modelado del Módulo C mediante las herramientas y notaciones de la Ingeniería de Software II: requerimientos funcionales, diagrama de casos de uso, diagrama de clases, diagramas de secuencia, diagrama de componentes y patrones de diseño. El alcance cubre el módulo de forma completa e incluye la documentación de las interfaces con los Módulos A, B y D del SIGE.

No forma parte del alcance la implementación en código fuente ni la integración tecnológica con los otros módulos, que se realizará en la etapa de consolidación del proyecto.

b) REQUERIMIENTOS FUNCIONALES DEL MÓDULO

Requerimientos propios del Módulo C

Los siguientes requerimientos funcionales han sido identificados a partir del análisis del dominio del problema y de las especificaciones del proyecto. Se presentan en orden de prioridad descendente.

RF01 — El sistema debe permitir registrar proveedores con sus datos completos, distinguiendo entre persona natural y persona jurídica, e incluyendo sus condiciones de pago negociadas.

RF02 — El sistema debe permitir crear órdenes de compra a partir de solicitudes generadas por el Módulo B (cuando el stock llega al mínimo) o de forma manual por el jefe de compras.

RF03 — El sistema debe permitir registrar recepciones de mercadería vinculadas a una orden de compra, admitiendo recepciones parciales en múltiples entregas.

RF04 — El sistema debe actualizar automáticamente el inventario en el Módulo B al confirmar una recepción, registrando el movimiento de entrada correspondiente.

RF05 — El sistema debe permitir ingresar facturas de compra emitidas por proveedores, vinculados a la orden de compra correspondiente y aplicando la tarifa de IVA vigente provista por el Módulo D.

RF06 — El sistema debe generar automáticamente un registro de cuenta por pagar al ingresar una factura de proveedor, con su fecha de vencimiento y monto pendiente calculado según las condiciones de pago.

RF07 — El sistema debe permitir registrar pagos a proveedores indicando el monto, la forma de pago (transferencia, cheque, efectivo) y la factura o cuenta por pagar asociada.

RF08 — El sistema debe actualizar automáticamente el saldo pendiente de cuentas por pagar al registrar cada pago, ya sea parcial o total.

RF09 — El sistema debe generar el estado de cuenta por proveedor, mostrando el historial de facturas, pagos realizados y saldo pendiente actualizado.

RF10 — El sistema debe alertar sobre pagos próximos a vencer en un horizonte configurable (por defecto: 7 días), identificando la factura, el proveedor y el monto pendiente.

RF11 — El sistema debe permitir consultar las órdenes de compra pendientes de recepción total, identificando las que tienen recepciones parciales o ninguna recepción registrada.

RF12 — El sistema debe generar reportes de obligaciones pendientes agrupados por proveedor, por fecha de vencimiento y por estado (pendiente, pagada_parcial, vencida).

RF13 — El sistema debe notificar automáticamente al Módulo B el ingreso de productos al inventario al confirmar una recepción de mercancía, especificando el producto, la cantidad recibida, la fecha y el identificador de la recepción, para que el Módulo B registre el movimiento de entrada y actualice el stock actual. Esta comunicación se realiza mediante la interfaz IF-03 , representada en el diseño como ServicioInventarioB.notificarMovimiento(), sin acceso directo a las tablas internas del Módulo B. Caso de uso asociado: CU-03 Registrar recepción de mercancía / CU-04 Actualizar inventario.

RF14 — El sistema debe consultar el catálogo de productos del Módulo B para seleccionar los ítems que se incluirán en una orden de compra, obteniendo código, descripción, unidad de medida y precio de referencia. (Interfaz IF-04 .) Caso de uso asociado: CU-02 Crear orden de compra / CU-13 Consultar catálogo de productos en Módulo B.

RF15 — El sistema debe recibir y procesar las solicitudes de compra generadas por el Módulo B cuando el stock de un producto alcanza o cae por debajo del mínimo establecido, permitiendo al Jefe de Compras convertirlas en órdenes de compra formales. (Interfaz IF-02 .) Caso de uso asociado: CU-02 Crear orden de compra.

RF16 — El sistema debe exponer el precio de costo de adquisición de los productos, registrado en las órdenes de compra, como un servicio de consulta de solo lectura para que el Módulo A pueda calcularlo como referencia al determinar márgenes de venta. (Interfaz IF-01.) Caso de uso asociado: CU-14 Exponer precio de costo.

RF17 — Al registrar una factura de compra, el sistema debe consultar al Módulo D la tarifa de IVA vigente y los porcentajes de retención aplicables según el tipo de proveedor y bien o servicio, aplicarlos automáticamente al calcular el total de la factura, y enviar al Módulo D los datos de la factura (proveedor, base imponible, tarifa de IVA y fecha) para que el Módulo D registre el IVA pagado y genere el comprobante de retención correspondiente. (Interfaces IF-06 , IC-D02 e IF-05 .) Caso de uso asociado: CU-05 Ingresar factura de proveedor.

Interfaces con otros módulos

Las interacciones con los demás módulos del SIGE han sido formalizadas como requerimientos funcionales y se documentan a continuación como referencia de trazabilidad:

- **RF13 (IF-03) — Módulo C → Módulo B:** Notificación de ingreso al inventario al confirmar recepción de mercancía.
- **RF14 (IF-04) — Módulo C → Módulo B:** Consulta del catálogo de productos para armar órdenes de compra.
- **RF15 (IF-02) — Módulo B → Módulo C:** Recepción de solicitudes de compra por stock mínimo.
- **RF16 (IF-01) — Módulo C → Módulo A:** Exposición de precios de costo de adquisición (solo lectura).
- **RF17 (IF-06 , IC-D02, IF-05) — Módulo C ↔ Módulo D:** Consulta de tarifa IVA vigente y retenciones; envío de datos de factura para registro de IVA pagado y generación del comprobante de retención.

Requerimientos no funcionales del Módulo C

Los siguientes requerimientos no funcionales rigen la calidad y comportamiento del módulo independientemente de la funcionalidad específica:

RNF01 — El tiempo de respuesta para operaciones de consulta (estado de cuenta, reportes de obligaciones) no debe superar los 3 segundos con hasta 50 registros simultáneos.

RNF02 — El módulo debe mantener la integridad referencial en todas las transacciones; cualquier fallo en la comunicación con los Módulos B o D debe registrarse en el log de sincronización sin perder los datos ya confirmados en el Módulo C.

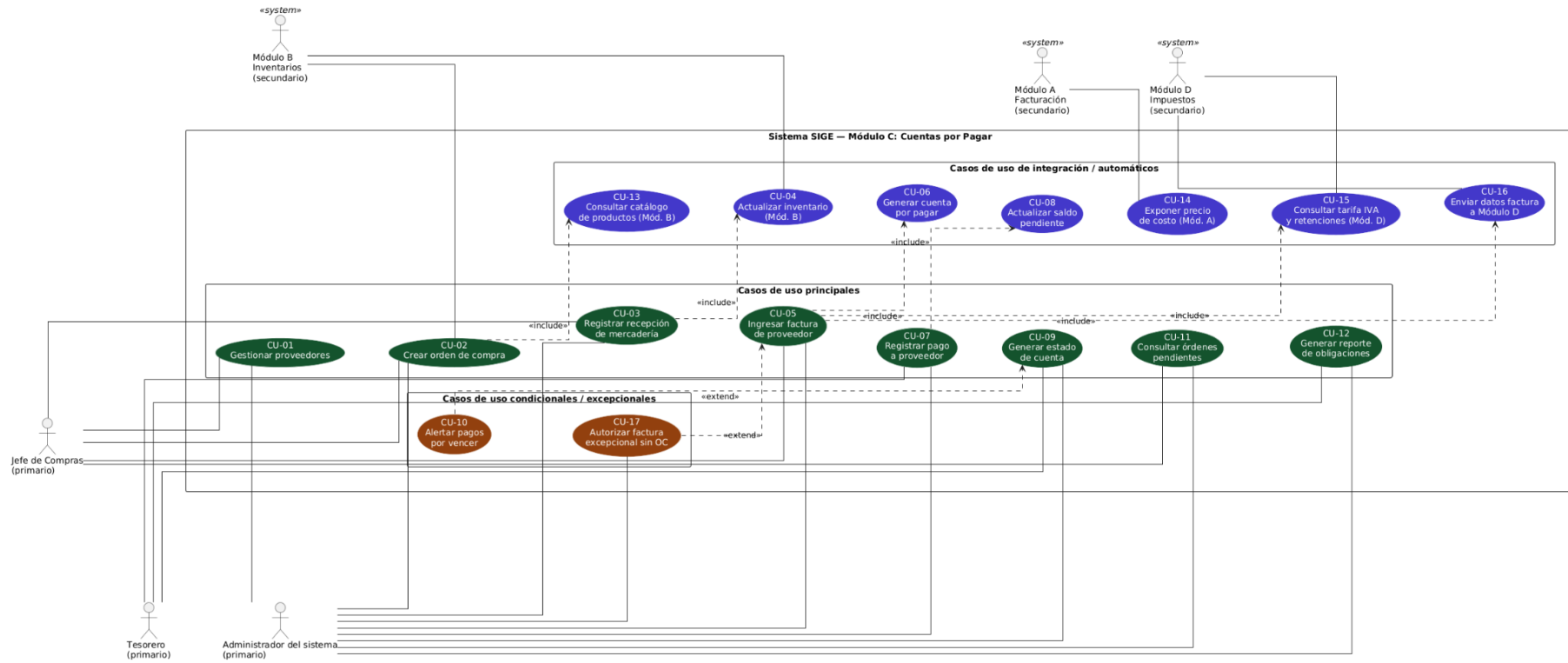
RNF03 — El acceso a las funciones del módulo debe estar controlado por roles: el Jefe de Compras solo puede gestionar el ciclo operativo (proveedores, órdenes, recepciones, facturas) y el Tesorero solo puede acceder al ciclo financiero (pagos, estado de cuenta, reportes). El administrador del sistema puede acceder a todas las funciones.

RNF04 — Toda operación que modifique datos financieros (registro de pago, ingreso de factura, actualización de saldo) debe quedar registrada en una bitácora de auditoría con usuario, fecha, hora y valores anterior y posterior.

RNF05 — Las interfaces de comunicación con los Módulos B y D deben implementarse como contratos definidos (interfaces en el sentido de la POO), de modo que un cambio

interno en cualquiera de esos módulos no requiera modificaciones en el Módulo C siempre que se respete el contrato.

c) Modelo funcional: Diagrama de casos de uso



Link:

<https://acortar.link/NnUyt2>

Descripción de Casos de Uso

CU-01: Gestionar proveedores

Actor primario: Jefe de Compras

Precondición: El usuario tiene permisos de administración de proveedores.

Postcondición: El proveedor queda registrado, modificado o desactivado en el sistema.

Flujo principal:

1. El Jefe de Compras accede al catálogo de proveedores y selecciona "Nuevo proveedor".
2. El sistema solicita los datos: razón social o nombre, tipo (persona natural o jurídica), RUC/cédula, dirección, teléfono, correo y condiciones de pago negociadas (plazo en días, forma de pago preferida).
3. El usuario completa los datos y confirma el registro.
4. El sistema valida que el RUC/cédula no esté duplicado y guarda el proveedor con estado "activo".

Flujo alternativo — modificación: En el paso 1, si el usuario selecciona un proveedor existente y elige "Editar", el sistema carga los datos actuales y permite modificarlos. Los cambios en condiciones de pago aplican únicamente a facturas futuras.

CU-02: Crear orden de compra

Actor primario: Jefe de Compras

Precondición: Existe al menos un proveedor activo y productos disponibles en el catálogo del Módulo B.

Postcondición: La orden de compra queda registrada en estado "emitida", lo que indica que está pendiente de recepción.

Flujo principal:

1. El Jefe de Compras accede al módulo de órdenes y selecciona "Nueva orden de compra".

2. El sistema permite seleccionar el origen: manual o a partir de una solicitud generada por el Módulo B (IF-02).
3. El usuario selecciona el proveedor y los productos a solicitar.
4. **[include]** El sistema ejecuta CU-13: consulta el catálogo de productos del Módulo B para obtener descripciones, unidades y precios de referencia.
5. El usuario ingresa las cantidades y precios acordados por ítem.
6. El sistema calcula el total y genera la orden con número correlativo, fecha y estado "emitida".

Flujo alternativo — desde solicitud del Módulo B: En el paso 2, si el origen es una solicitud automática, el sistema precarga el producto y la cantidad sugerida; el usuario puede ajustarlos antes de confirmar.

CU-03: Registrar recepción de mercadería

Actor primario: Jefe de Compras

Precondición: Existe una orden de compra en estado "emitida" o "recibida parcialmente".

Postcondición: La recepción queda registrada y el inventario del Módulo B se actualiza automáticamente.

Flujo principal:

1. El Jefe de Compras accede al módulo de recepciones y busca la orden de compra correspondiente.
2. El sistema muestra el detalle de la orden: ítems esperados, cantidades y cantidades previamente recibidas.
3. El usuario registra las cantidades efectivamente recibidas en esta entrega y la fecha de recepción.
4. El sistema valida que las cantidades no superen las pendientes de recepción.
5. El sistema guarda la recepción y, mediante la notificación de observadores, actualiza el estado de la orden de compra a "recibida_parcial" o "recibida_total", según corresponda.
6. **[include]** El sistema ejecuta CU-04: notifica al Módulo B el ingreso de productos para actualizar el stock.

Flujo alternativo — recepción con diferencias: Si las cantidades recibidas son menores a las esperadas, el sistema registra la diferencia y mantiene la orden en estado "recibida parcialmente" para futuras entregas.

CU-04: Actualizar inventario (incluido)

Actor: Sistema (disparado automáticamente desde CU-03)

Precondición: Se ha confirmado una recepción de mercadería en el Módulo C.

Postcondición: El stock del producto en el Módulo B refleja las unidades recién ingresadas.

Flujo principal:

1. Al confirmar la recepción, el Módulo C genera una notificación hacia el Módulo B con el código de producto, cantidad recibida, fecha e identificador de la recepción mediante la interfaz IF-03 .
2. El Módulo B procesa la notificación recibida mediante `ServicioInventarioB.notificarMovimiento()`, registra el movimiento de entrada en inventario y actualiza el stock actual del producto.
3. El Módulo B devuelve una confirmación de actualización al Módulo C.
4. El Módulo C marca la recepción como notificada mediante el atributo `notificado_inventario`.

Flujo alternativo — fallo de comunicación: Si el Módulo B no confirma la actualización, el sistema registra el error en un log de sincronización y notifica al administrador para resolución manual.

CU-05: Ingresar factura de proveedor

Actor primario: Jefe de Compras

Precondición: Existe una orden de compra en estado "recibida" o "recibida parcialmente".

Postcondición: La factura queda registrada, se envían sus datos tributarios al Módulo D y se genera automáticamente la cuenta por pagar asociada.

Flujo principal:

1. El Jefe de Compras accede al módulo de facturas y selecciona "Nueva factura de proveedor".
2. El sistema solicita: número de factura, fecha de emisión, proveedor, orden de compra asociada, base imponible, tipo de bien o servicio y datos tributarios requeridos.
3. El usuario vincula la factura a una orden de compra existente.
4. El sistema valida que el monto no supere el valor pendiente de facturación de la orden.
5. [include] El sistema ejecuta CU-15: consulta al Módulo D la tarifa IVA vigente y las reglas de retención aplicables.
6. El sistema calcula el total de la factura con base imponible, IVA y valores tributarios correspondientes.
7. El sistema registra la factura con estado "pendiente_retencion".
8. [include] El sistema ejecuta CU-16: envía los datos de la factura al Módulo D para registrar el IVA pagado y generar el comprobante de retención.

9. [include] El sistema ejecuta CU-06: genera automáticamente el registro de cuenta por pagar.

Flujo Alternativo — Factura sin orden asociada: 1. Si en el paso 3 el usuario indica que no existe una orden de compra vinculada, el sistema bloquea el flujo regular. 2. El sistema solicita de forma obligatoria las credenciales de validación y autorización del **Gerente General** (Ejecuta CU-17: Autorizar factura excepcional sin OC). 3. Si el **Gerente General** aprueba la excepción, el sistema permite continuar con el registro de la factura; de lo contrario, la operación se cancela.

CU-06: Generar cuenta por pagar (incluido)

Actor: Sistema (disparado automáticamente desde CU-05)

Precondición: Se ha registrado una factura de proveedor válida.

Postcondición: Existe un registro de cuenta por pagar con fecha de vencimiento y saldo inicial calculados.

Flujo principal:

1. El sistema obtiene las condiciones de pago del proveedor (plazo en días).
2. Calcula la fecha de vencimiento sumando el plazo a la fecha de emisión de la factura.
3. Crea el registro de CuentaPorPagar con monto pendiente igual al total de la factura, estado "pendiente" y la fecha de vencimiento calculada.
4. Asocia el registro a la factura y al proveedor correspondientes.

CU-07: Registrar pago a proveedor

Actor primario: Tesorero

Precondición: Existe al menos una cuenta por pagar con saldo mayor a cero.

Postcondición: El pago queda registrado y el saldo de la cuenta por pagar se actualiza.

Flujo principal:

1. El Tesorero accede al módulo de pagos y busca al proveedor o la factura específica.
2. El sistema muestra las cuentas por pagar vigentes con saldo pendiente y fecha de vencimiento.
3. El usuario selecciona la cuenta, ingresa el monto a abonar y la forma de pago (transferencia, cheque o efectivo).
4. El sistema valida que el monto no supere el saldo pendiente.
5. El sistema registra el pago con fecha, hora y comprobante.
6. [include] El sistema ejecuta CU-08: recalcula y actualiza el saldo pendiente.

Flujo alternativo — pago parcial: Si el monto es menor al saldo total, el sistema acepta el abono, mantiene la cuenta en estado "pagada_parcial" y conserva la fecha de vencimiento original para el saldo restante.

CU-08: Actualizar saldo pendiente (incluido)

Actor: Sistema (disparado automáticamente desde CU-07)

Precondición: Se ha registrado un pago asociado a una cuenta por pagar.

Postcondición: El saldo de la cuenta por pagar refleja el nuevo monto pendiente.

Flujo principal:

1. El sistema localiza la cuenta por pagar asociada al pago registrado.
2. Resta el monto pagado del saldo pendiente actual.
3. Si el nuevo saldo es cero, cambia el estado de la cuenta a "pagada" y registra la fecha de cancelación.
4. Si el saldo es mayor a cero, actualiza el estado a "pagada_parcial".

CU-09: Generar estado de cuenta por proveedor

Actor primario: Tesorero

Precondición: El proveedor seleccionado tiene al menos una transacción registrada (factura o pago).

Postcondición: El sistema presenta o exporta el estado de cuenta con el historial completo del proveedor.

Flujo principal:

1. El Tesorero selecciona el proveedor y el período de consulta (fechas desde/hasta).
2. El sistema recopila todas las facturas emitidas, pagos realizados y saldos parciales del período.
3. El sistema presenta el estado de cuenta: fecha de cada transacción, tipo (factura o pago), monto y saldo acumulado.
4. **[extend]** Si existen facturas próximas a vencer dentro del horizonte configurado, el sistema ejecuta CU-10 y muestra las alertas al final del reporte.
5. El Tesorero puede exportar el estado de cuenta en formato PDF o Excel.

CU-10: Alertar pagos próximos a vencer (extendido)

Actor: Sistema (se extiende desde CU-09 de forma condicional)

Precondición: Existen cuentas por pagar con fecha de vencimiento dentro del horizonte configurado (por defecto 7 días) y con saldo pendiente mayor a cero.

Postcondición: El sistema notifica al Tesorero las obligaciones próximas a vencer.

Flujo principal:

1. El sistema consulta todas las cuentas por pagar con estado "pendiente" o "pagada_parcial".
2. Filtra aquellas cuya fecha de vencimiento esté dentro del horizonte configurado.
3. Genera una lista de alertas con: nombre del proveedor, número de factura, fecha de vencimiento y monto pendiente.
4. Presenta las alertas en pantalla con indicación visual de urgencia (vence hoy, en menos de 3 días, en menos de 7 días).

Condición de extensión: Este caso de uso solo se activa si la consulta del estado de cuenta (CU-09) detecta facturas dentro del horizonte de alerta; de lo contrario no se ejecuta.

CU-11: Consultar órdenes de compra pendientes

Actor primario: Jefe de Compras

Precondición: Existen órdenes de compra registradas en el sistema.

Postcondición: El sistema presenta el listado filtrado de órdenes con recepción incompleta o ausente.

Flujo principal:

1. El Jefe de Compras accede al módulo de órdenes y selecciona "Órdenes pendientes de recepción".
2. El sistema recupera todas las órdenes con estado "emitida" o "recibida parcialmente".
3. El sistema presenta el listado con: número de orden, proveedor, fecha de emisión, porcentaje de recepción completado y días transcurridos desde la emisión.
4. El usuario puede filtrar por proveedor, rango de fechas o estado.
5. Al seleccionar una orden, el sistema muestra el detalle de los ítems pendientes de recepción.

CU-12: Generar reporte de obligaciones pendientes

Actor primario: Tesorero

Precondición: Existen cuentas por pagar registradas en el sistema.

Postcondición: El sistema presenta o exporta el reporte agrupado según el criterio seleccionado.

Flujo principal:

1. El Tesorero selecciona los criterios de filtro (rango de fechas, proveedor o estado de la obligación).
2. El sistema consulta la base de datos y calcula los saldos pendientes de las Cuentas por Pagar correspondientes.
3. El sistema genera un reporte detallado en pantalla con opción de exportación a PDF o Excel.

CU-13: Consultar catálogo de productos (incluido)

Actor: Sistema (disparado automáticamente desde CU-02)

Precondición: El Módulo B tiene productos registrados en su catálogo.

Postcondición: El Módulo C dispone de la lista de productos disponibles para incluir en la orden de compra.

Flujo principal:

1. Al crear una orden de compra, el sistema realiza una consulta de solo lectura al catálogo del Módulo B (interfaz IF-04).
2. El Módulo B devuelve la lista de productos con código, descripción, unidad de medida y precio de referencia.
3. El sistema presenta los productos al usuario para su selección dentro del formulario de la orden de compra.

CU-14: Exponer precio de costo

Actor Primario/Secundario: Módulo A — Facturación.

Precondición: Existen órdenes de compra con precios de adquisición registrados.

Postcondición: El Módulo A obtiene el precio de costo de adquisición solicitado en modo de solo lectura.

Flujo principal:

1. El Módulo A solicita al Módulo C el precio de costo de un producto específico mediante la interfaz de integración técnica (Interfaz **IF-01 / IC-A01**).
2. El Módulo C consulta de forma automática el último precio unitario pactado y registrado en las órdenes de compra confirmadas para ese ítem.
3. El sistema devuelve el valor del precio de costo como dato de solo lectura, sin permitir ningún tipo de modificación sobre los registros del Módulo C.

CU-15: Consultar tarifa IVA y retenciones

Actor Principal: Sistema / Módulo D — Impuestos.

Precondición: El Jefe de Compras está ingresando una factura de proveedor.

Postcondición: El Módulo C obtiene la tarifa IVA vigente y los porcentajes de retención aplicables para el cálculo tributario.

Flujo Principal:

1. Durante el proceso de ingreso de la factura (CU-05), el Módulo C solicita de forma automática al Módulo D la tarifa IVA vigente.
2. El Módulo D devuelve la tarifa aplicable y los porcentajes de retención correspondientes según el tipo de proveedor y el tipo de bien o servicio seleccionado.
3. El Módulo C utiliza de inmediato esos valores para calcular de forma exacta los impuestos y retenciones de la factura.

CU-16: Enviar datos factura a Módulo D

Actor Principal: Sistema / Módulo D — Impuestos.

Precondición: La factura de proveedor ha sido registrada con datos tributarios válidos en el sistema.

Postcondición: El Módulo D recibe los datos de la factura para registrar el IVA pagado y generar el comprobante de retención oficial.

Flujo Principal:

1. Tras confirmar el guardado, el Módulo C envía al Módulo D los datos clave de la factura: proveedor, base imponible, valor del IVA, tipo de bien o servicio y fecha de emisión.
2. El Módulo D procesa la información, registra el IVA pagado en sus libros y genera el comprobante de retención.
3. El Módulo D devuelve al Módulo C el número de autorización oficial del SRI y el archivo XML de la retención generado.
4. El Módulo C actualiza los datos internos de retención de la factura y da por finalizado el proceso de integración.

Resumen de relaciones: Las relaciones «include» CU-04, CU-06, CU-08, CU-13, CU-15 y CU-16 representan comportamientos que el sistema ejecuta como parte obligatoria del flujo base del caso de uso origen. Por otro lado, el CU-10 se modela como «extend» debido a que solo se ejecuta cuando existen pagos próximos a vencer dentro del horizonte temporal configurado. Finalmente, el CU-14 se mantiene como un caso de uso independiente asociado directamente al Módulo A, ya que corresponde a una consulta externa de precio de costo y no a la generación del reporte interno de obligaciones.

CU-17: Autorizar factura excepcional sin OC

Actor Principal: Administrador del sistema.

Precondición: El jefe de Compras intenta registrar una factura de proveedor sin una orden de compra asociada.

Postcondición: La factura excepcional queda autorizada para continuar su registro o la operación quedará cancelada si la autorización es rechazada.

Flujo Principal:

1. Durante el ingreso de la factura del proveedor, el sistema detecta que no existe una orden de compra asociada.
2. El sistema bloquea temporalmente el flujo regular de registro y solicita autorización del Administrador del sistema.
3. El Administrador del sistema revisa los datos de la factura, el proveedor, el motivo de la excepción y los valores ingresados.
4. Si la información es válida, el Administrador del sistema aprueba la autorización excepcional.
5. El sistema registra la autorización, permite continuar con el flujo de CU-05 y deja constancia de la aprobación en la bitácora de auditoría.

Flujo Alternativo — autorización rechazada:

Si el Administrador del sistema rechaza la solicitud, el sistema cancela el registro de la factura sin orden de compra asociada y notifica al jefe de Compras que la operación no puede continuar.

Condición de extensión:

Este caso de uso se ejecuta como «extend» de CU-05 Ingresar factura de proveedor, únicamente cuando la factura no tiene una orden de compra asociada.

Requerimiento	Caso de Uso	Cobertura	Estado
RF01 — Registrar proveedores	CU-01: Gestionar proveedores	Registro completo de proveedores, distinguiendo persona natural o jurídica e incluyendo condiciones de pago negociadas.	Cubierto
RF02 — Crear órdenes de compra	CU-02: Crear orden de compra / CU-13: Consultar catálogo	Permite crear órdenes de compra manuales o desde solicitudes generadas por el Módulo B; consulta productos mediante IF-04.	Cubierto

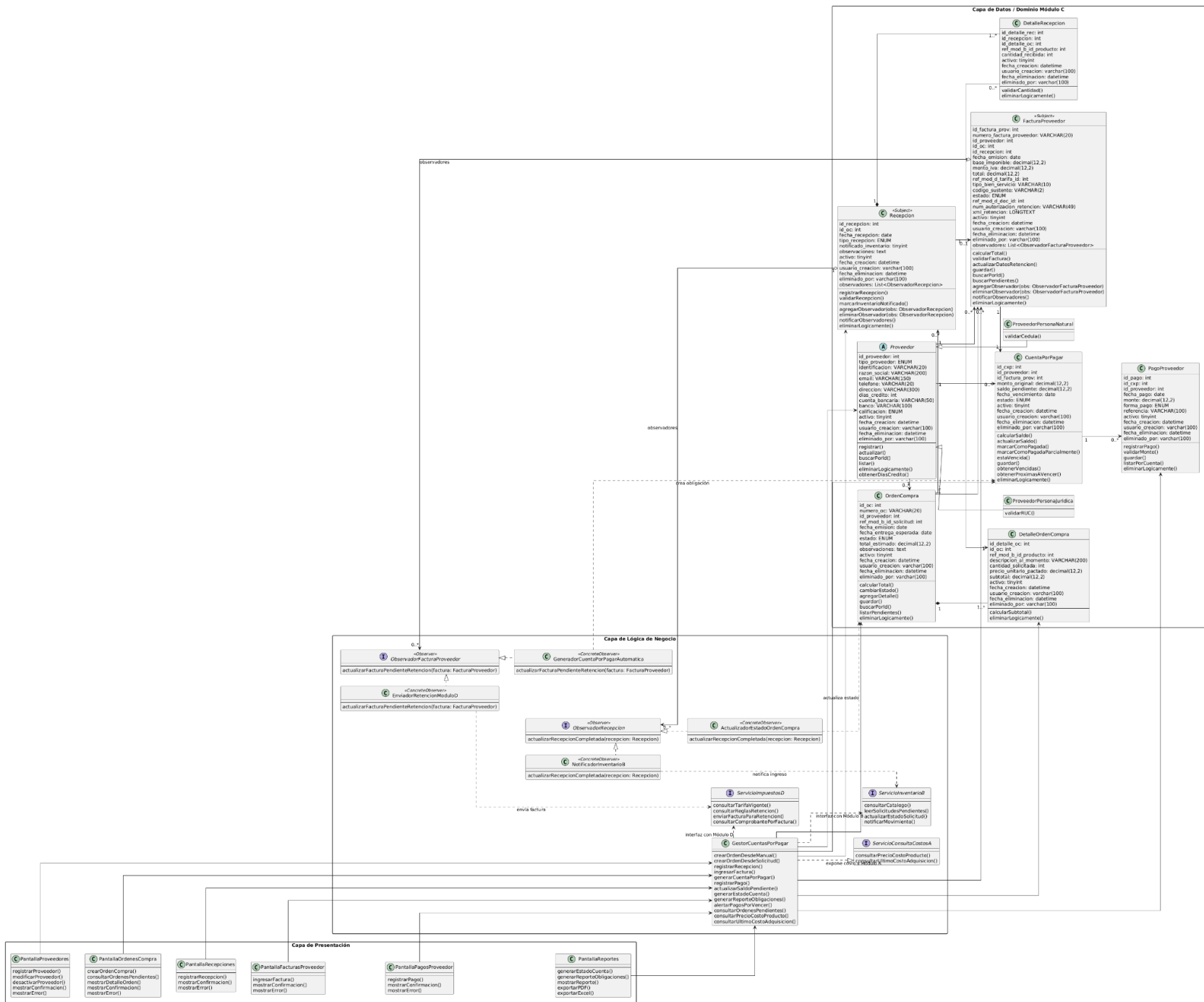
RF03 — Registrar recepciones parciales	CU-03: Registrar recepción de mercadería	Permite registrar recepciones vinculadas a órdenes de compra, admitiendo entregas múltiples y parciales.	Cubierto
RF04 — Actualizar inventario Módulo B	CU-04: Actualizar inventario	Actualiza el inventario del Módulo B al confirmar una recepción, registrando el movimiento de entrada correspondiente mediante integración.	Cubierto
RF05 — Ingresar facturas proveedor	CU-05: Ingresar factura de proveedor / CU-15 / CU-16	Registra facturas de proveedor vinculadas a una orden de compra, consulta IVA y retenciones, y envía datos tributarios al Módulo D.	Cubierto
RF06 — Generar cuenta por pagar	CU-06: Generar cuenta por pagar	Genera automáticamente una cuenta por pagar al registrar una factura válida, calculando vencimiento y monto pendiente.	Cubierto
RF07 — Registrar pagos	CU-07: Registrar pago a proveedor	Registra pagos a proveedores indicando monto, forma de pago y cuenta por pagar asociada.	Cubierto
RF08 — Actualizar saldo pendiente	CU-08: Actualizar saldo pendiente	Actualiza automáticamente el saldo pendiente de la cuenta por pagar después de cada pago, sea parcial o total.	Cubierto
RF09 — Estado de cuenta por proveedor	CU-09: Generar estado de cuenta	Presenta el historial de facturas, pagos realizados y saldo pendiente actualizado por proveedor.	Cubierto
RF10 — Alertar pagos por vencer	CU-10: Alertar pagos próximos a vencer	Genera alertas de obligaciones próximas a vencer dentro de un horizonte configurable, por defecto 7 días.	Cubierto
RF11 — Consultar OC pendientes	CU-11: Consultar órdenes pendientes	Identifica órdenes de compra con recepción parcial o sin recepción registrada.	Cubierto

RF12 — Reporte obligaciones pendientes	CU-12: Generar reporte de obligaciones	Genera reportes de obligaciones pendientes agrupadas por proveedor, fecha de vencimiento y estado.	Cubierto
RF13 — Notificar ingreso inventario	CU-03: Registrar recepción / CU-04: Actualizar inventario	Notifica al Módulo B el ingreso de productos mediante IF-03 / ServicioInventarioB.notificarMovimiento(), sin trigger ni acceso directo a tablas internas.	Cubierto
RF14 — Consultar catálogo Módulo B	CU-13: Consultar catálogo de productos	Consulta el catálogo de productos del Módulo B mediante IF-04 para seleccionar ítems de la orden de compra.	Cubierto
RF15 — Solicitud compra Módulo B	CU-02: Crear orden de compra	Procesa solicitudes de compra generadas por el Módulo B cuando el stock llega al mínimo, mediante IF-02.	Cubierto
RF16 — Exponer precio de costo	CU-14: Exponer precio de costo	Expone el precio de costo de adquisición al Módulo A mediante IF-01 / ServicioConsultaCostosA, en modo solo lectura.	Cubierto
RF17 — Gestión IVA y retenciones	CU-05: Ingresar factura / CU-15: Consultar tarifa IVA y retenciones / CU-16: Enviar datos factura a Módulo D	Consulta tarifa IVA y retenciones mediante IF-06 / IC-D02, aplica los valores al cálculo de factura y envía los datos al Módulo D mediante IF-05.	Cubierto
RNF03 — Control de acceso por roles	CU-17: Autorizar factura excepcional sin OC	Controla permisos por rol y permite que el Administrador del sistema autorice excepcionalmente una factura sin orden de compra asociada.	Cubierto

d) Modelo estructural: Diagrama de clases

Link:

<https://acortar.link/b8Lhj1>



Explicación completa del diagrama de clases

El diagrama de clases del Módulo C — Cuentas por Pagar representa la estructura interna del subsistema encargado de administrar el ciclo completo de compras y obligaciones financieras de la empresa. El diseño fue desarrollado siguiendo la arquitectura acordada para el proyecto SIGE: un monolito modular por capas, compuesto por una capa de presentación, una capa de lógica de negocio y una capa de datos o dominio.

Esta arquitectura busca separar responsabilidades dentro del sistema para facilitar el mantenimiento, la escalabilidad y la integración entre módulos. Además, la comunicación entre módulos no se realiza mediante acceso directo a tablas, sino a través de interfaces y contratos definidos, reduciendo el acoplamiento entre componentes del SIGE.

En la versión actualizada del diagrama también se incorpora el patrón de diseño Observer, aplicado en dos procesos críticos del Módulo C: la recepción de mercadería y el registro de facturas de proveedor. Esta adición permite que ciertas acciones automáticas se ejecuten cuando ocurre un evento importante, sin que las clases principales queden acopladas directamente a todos los procesos secundarios que deben reaccionar a dicho evento.

Arquitectura por capas

El diagrama se organiza en tres capas principales:

Capa de Presentación

La capa de presentación representa la interfaz visible para el usuario. Aquí se ubican todas las pantallas mediante las cuales el usuario interactúa con el módulo.

Estas clases no contienen reglas de negocio ni acceden directamente a la base de datos. Su función es recibir información del usuario, mostrar resultados y enviar solicitudes hacia la capa lógica.

En el diagrama actualizado, las pantallas incluyen operaciones básicas asociadas a la interacción del usuario, como registrarProveedor(), crearOrdenCompra(), registrarRecepcion(), ingresarFactura(), registrarPago(), generarEstadoCuenta(), generarReporteObligaciones(), mostrarConfirmacion() y mostrarError(). Estas operaciones no ejecutan reglas de negocio, sino que representan las acciones de entrada y salida de la interfaz antes de delegar el procesamiento al GestorCuentasPorPagar.

Las clases de esta capa son:

PantallaProveedores

Representa la interfaz utilizada para administrar proveedores.

Desde esta pantalla el Jefe de Compras puede registrar nuevos proveedores, modificar información existente, consultar proveedores registrados y desactivar proveedores mediante borrado lógico.

Esta pantalla está asociada directamente al caso de uso CU-01 Gestionar proveedores.

Su función es capturar los datos ingresados por el usuario y enviarlos al GestorCuentasPorPagar para que se ejecuten las validaciones y operaciones correspondientes.

PantallaOrdenesCompra

Representa la interfaz donde se crean y consultan órdenes de compra.

Desde esta pantalla el usuario puede crear órdenes manuales, crear órdenes desde solicitudes enviadas por el Módulo B, consultar órdenes pendientes de recepción y visualizar detalles de órdenes emitidas.

Esta pantalla participa en los casos de uso CU-02 Crear orden de compra y CU-11 Consultar órdenes pendientes.

La pantalla no calcula precios ni valida lógica financiera; solamente muestra formularios y envía la información a la capa lógica.

PantallaRecepciones

Representa la interfaz utilizada para registrar recepciones de mercadería.

Desde esta pantalla el usuario puede buscar órdenes de compra, registrar cantidades recibidas, registrar recepciones parciales o completas y confirmar el ingreso de productos.

Está asociada al caso de uso CU-03 Registrar recepción de mercadería.

Cuando el usuario confirma la recepción, la pantalla envía la solicitud al GestorCuentasPorPagar, el cual coordina el registro de la recepción. Una vez que la recepción queda registrada y validada, la clase Recepcion actúa como sujeto del patrón Observer y notifica a los observadores correspondientes para ejecutar acciones automáticas, como notificar al Módulo B el ingreso de mercadería y modificar el estado de la orden de compra.

PantallaFacturasProveedor

Representa la interfaz donde se ingresan las facturas emitidas por proveedores.

Permite registrar datos tributarios, asociar facturas a órdenes de compra, consultar tarifas IVA, validar montos de facturación y generar automáticamente cuentas por pagar.

Está relacionada con el caso de uso CU-05 Ingresar factura de proveedor.

Esta pantalla es importante porque inicia la integración con el Módulo D — Impuestos. En el diagrama actualizado, cuando una factura queda registrada en estado pendiente_retencion, es decir, pendiente de retención, la clase FacturaProveedor actúa como sujeto del patrón Observer y notifica a los observadores encargados de enviar la factura al Módulo D y generar automáticamente la cuenta por pagar.

PantallaPagosProveedor

Representa la interfaz utilizada por el Tesorero para registrar pagos.

Desde esta pantalla el usuario puede consultar cuentas pendientes, registrar pagos parciales o totales, seleccionar forma de pago y consultar saldos pendientes.

Está asociada al caso de uso CU-07 Registrar pago a proveedor.

La actualización real de saldos ocurre en la lógica del sistema y no directamente desde la pantalla.

PantallaReportes

Representa la interfaz destinada a reportes y consultas financieras.

Permite generar estados de cuenta, consultar obligaciones pendientes, visualizar pagos próximos a vencer y exportar reportes.

Participa en los casos de uso CU-09 Generar estado de cuenta, CU-10 Alertar pagos por vencer y CU-12 Generar reporte de obligaciones.

Capa de Lógica de Negocio

Esta capa contiene las reglas y procesos principales del sistema.

Su función es validar operaciones, coordinar procesos, aplicar reglas financieras, gestionar estados, integrar módulos externos y ejecutar procesos automáticos mediante observadores.

GestorCuentasPorPagar

Es la clase controladora principal del módulo.

Implementa el patrón Controller de GRASP y centraliza la coordinación de todos los procesos importantes del sistema.

Sus responsabilidades incluyen crear órdenes de compra, registrar recepciones, ingresar facturas, generar cuentas por pagar, registrar pagos, actualizar saldos, generar reportes, generar alertas de vencimiento y consultar órdenes pendientes.

Esta clase no almacena datos directamente. Su función es coordinar las entidades del dominio y las interfaces externas.

El GestorCuentasPorPagar actúa como puente entre la capa de presentación, la capa de dominio y los servicios externos de Inventarios e Impuestos.

Además, el GestorCuentasPorPagar incorpora operaciones de consulta de costo, como consultarPrecioCostoProducto() y consultarUltimoCostoAdquisicion(), que permiten responder a solicitudes de solo lectura realizadas por el Módulo A a través de la interfaz ServicioConsultaCostosA.

En la versión actualizada, el gestor sigue siendo el coordinador principal, pero ya no concentra por sí solo todas las reacciones automáticas posteriores a una recepción o una factura. Esas acciones se delegan mediante el patrón Observer, lo que permite separar mejor las responsabilidades y reducir el acoplamiento entre procesos.

El método generarCuentaPorPagar() del gestor representa la coordinación del caso de uso CU-06, mientras que GeneradorCuentaPorPagarAutomatica ejecuta la creación concreta de la cuenta por pagar como observador de FacturaProveedor.

ServicioInventarioB

Es una interfaz de integración con el Módulo B — Inventarios.

La comunicación entre módulos no se realiza accediendo directamente a tablas externas. En su lugar, se utilizan contratos definidos mediante interfaces.

Esta interfaz permite consultar el catálogo de productos, leer solicitudes de compra, actualizar el estado de solicitudes y notificar movimientos de inventario.

Esto implementa los requerimientos relacionados con la integración del Módulo C con Inventarios, especialmente la consulta de productos, lectura de solicitudes de compra y notificación de ingreso de mercadería.

El objetivo es desacoplar el Módulo C del diseño interno del Módulo B.

ServicioImpuestosD

Es la interfaz de integración con el Módulo D — Impuestos.

Permite consultar tarifas IVA, consultar reglas de retención, enviar facturas para retención y consultar comprobantes tributarios.

Esta interfaz evita que el Módulo C dependa directamente de las tablas tributarias del Módulo D. El Módulo C solo conoce el contrato definido por la interfaz, no la implementación interna del módulo tributario.

ServicioConsultaCostosA

Es una interfaz de integración expuesta por el Módulo C para el Módulo A — Facturación.

Su función es permitir la consulta de precios de costo de adquisición de los productos en modo solo lectura, de acuerdo con el requerimiento RF16 y el caso de uso CU-14.

A diferencia de ServicioInventarioB y ServicioImpuestosD, que representan servicios consumidos por el Módulo C, ServicioConsultaCostosA representa un contrato que el Módulo C ofrece al Módulo A. Mediante esta interfaz, el Módulo A puede obtener el último costo de adquisición registrado en las órdenes de compra sin acceder directamente a las tablas internas del Módulo C.

ObservadorRecepcion

Es una interfaz que representa el Observer asociado al proceso de recepción de mercadería.

Define la operación actualizarRecepcionComplelada(recepcion: Recepcion).

Esta operación se ejecuta cuando una recepción ha sido registrada y completada correctamente. Su objetivo es permitir que varios componentes reaccionen ante el evento sin que Recepcion tenga que conocer los detalles internos de cada acción posterior.

NotificadorInventarioB

Es un observador concreto de Recepcion.

Su responsabilidad es reaccionar cuando una recepción se completa y notificar al Módulo B — Inventarios sobre el ingreso de productos. Para ello utiliza la interfaz ServicioInventarioB, enviando la información necesaria para que el Módulo B registre el movimiento de entrada y actualice su propio stock.

Esta clase representa la automatización del flujo de notificación de inventario después de registrar una recepción de mercadería.

ActualizadorEstadoOrdenCompra

Es otro observador concreto de Recepcion.

Su responsabilidad es actualizar el estado de la OrdenCompra asociada a la recepción. Dependiendo de las cantidades recibidas, la orden puede pasar a un estado como recibida_parcial o recibida_total.

Esta clase permite separar la lógica de actualización de la orden respecto del registro de la recepción, evitando que Recepcion concentre más responsabilidades de las necesarias.

ObservadorFacturaProveedor

Es una interfaz que representa el Observer asociado al proceso de facturación de proveedores.

Define la operación actualizarFacturaPendienteRetencion(factura: FacturaProveedor).

Esta operación se ejecuta cuando una factura ha sido registrada y queda en estado pendiente_retencion. Su objetivo es permitir que varios procesos automáticos reaccionen al registro de la factura sin acoplar directamente FacturaProveedor con todas las acciones posteriores.

EnviadorRetencionModuloD

Es un observador concreto de FacturaProveedor.

Su responsabilidad es tomar los datos de la factura registrada y enviarlos al Módulo D — Impuestos mediante la interfaz ServicioImpuestosD. De esta forma, el Módulo D puede registrar el IVA pagado y generar el comprobante de retención correspondiente.

Esta clase representa la integración tributaria automática del módulo.

GeneradorCuentaPorPagarAutomatica

Es otro observador concreto de FacturaProveedor.

Su responsabilidad es generar automáticamente la CuentaPorPagar asociada a la factura registrada. Para ello toma la información de la factura, el proveedor y las condiciones de crédito correspondientes.

Esta clase permite que la creación de la obligación financiera quede automatizada como una reacción al evento de registro de factura, sin que FacturaProveedor tenga que encargarse directamente de toda la lógica financiera.

Capa de Datos / Dominio

Esta capa contiene las entidades principales del negocio.

Cada clase representa una tabla o entidad importante del proceso de compras y pagos.

Las entidades contienen atributos, relaciones y operaciones propias del dominio.

En esta capa se mantienen las clases principales del modelo original, pero se han incorporado métodos y relaciones necesarias para representar el patrón Observer en Recepcion y FacturaProveedor.

Explicación de las entidades

Proveedor

Es una clase abstracta que representa cualquier proveedor registrado en el sistema.

Contiene información general como identificación, razón social, correo, dirección, cuenta bancaria y días de crédito.

Se especializa en dos subclases: `ProveedorPersonaNatural` y `ProveedorPersonaJuridica`.

Esto permite manejar validaciones tributarias diferentes.

ProveedorPersonaNatural

Representa proveedores que operan como personas naturales.

Incluye validaciones específicas para cédula ecuatoriana.

ProveedorPersonaJuridica

Representa proveedores constituidos legalmente como empresas.

Incluye validaciones específicas para RUC.

OrdenCompra

Representa la orden formal emitida al proveedor.

Contiene número de orden, proveedor asociado, fechas, estado y total estimado.

Puede originarse manualmente o desde solicitudes automáticas del Módulo B.

Sus estados representan el avance del proceso de compra: borrador, emitida, recibida_parcial, recibida_total y anulada.

En el diagrama actualizado, `OrdenCompra` también se relaciona con el observador `ActualizadorEstadoOrdenCompra`, ya que este componente se encarga de modificar el estado de la orden cuando se registra una recepción de mercadería.

DetalleOrdenCompra

Representa cada producto solicitado dentro de una orden.

Contiene producto referenciado, cantidad solicitada, precio pactado y subtotal.

La relación con `OrdenCompra` es de composición porque un detalle no puede existir sin su orden.

Además, esta clase permite conservar el precio unitario pactado de adquisición, dato que posteriormente puede ser consultado como referencia de costo por el Módulo A mediante la interfaz `ServicioConsultaCostosA`.

Recepcion

Representa el ingreso físico de mercadería.

Permite registrar recepciones completas y recepciones parciales.

Una orden puede tener varias recepciones.

También controla si ya se notificó al Módulo B para actualizar el inventario mediante el atributo `notificado_inventario`.

En la versión actualizada del diagrama, Recepcion cumple el rol de Subject dentro del patrón Observer. Esto significa que mantiene una lista de observadores y los notifica cuando ocurre un evento importante, específicamente cuando la recepción ha sido registrada y completada.

Por esta razón, se agregan los siguientes elementos:

observadores: List<ObservadorRecepcion>

agregarObservador(obs: ObservadorRecepcion)

eliminarObservador(obs: ObservadorRecepcion)

notificarObservadores()

Con esto, Recepcion no necesita ejecutar directamente todas las acciones posteriores al registro. En su lugar, notifica a sus observadores. Los observadores concretos reaccionan de forma independiente:

NotificadorInventarioB notifica al Módulo B el ingreso de mercadería mediante ServicioInventarioB.notificarMovimiento().

ActualizadorEstadoOrdenCompra actualiza el estado de la OrdenCompra asociada.

Esta aplicación del patrón reduce el acoplamiento y permite agregar nuevos comportamientos en el futuro sin modificar directamente la clase Recepcion.

DetalleRecepcion

Representa cada producto recibido en una recepción.

Se relaciona con DetalleOrdenCompra para comparar cantidad solicitada y cantidad recibida.

Esto permite controlar diferencias y recepciones parciales.

FacturaProveedor

Representa la factura emitida por el proveedor.

Contiene datos tributarios, base imponible, IVA, total, estado e información de retención.

Esta entidad inicia la integración con el Módulo D.

Cuando se registra una factura, se consulta IVA, se aplican reglas de retención, se envían datos al módulo tributario y se genera la cuenta por pagar.

En la versión actualizada del diagrama, FacturaProveedor cumple el rol de Subject dentro del patrón Observer. Esto significa que puede notificar a otros componentes cuando la factura queda registrada en estado pendiente_retencion, es decir, pendiente de retención.

Por esta razón, se agregan los siguientes elementos:

observadores: List<ObservadorFacturaProveedor>

agregarObservador(obs: ObservadorFacturaProveedor)

eliminarObservador(obs: ObservadorFacturaProveedor)

notificarObservadores()

Los observadores asociados a FacturaProveedor son EnviadorRetencionModuloD y GeneradorCuentaPorPagarAutomatica.

EnviadorRetencionModuloD se encarga de enviar la factura al Módulo D mediante ServicioImpuestosD para que se genere el comprobante de retención. Luego de recibir el comprobante de retención, la factura puede actualizarse al estado retencion_generada.

GeneradorCuentaPorPagarAutomatica se encarga de crear la CuentaPorPagar asociada a la factura registrada.

Gracias a esta estructura, FacturaProveedor no queda acoplada directamente a todos los procesos posteriores. Solamente emite la notificación, y cada observador ejecuta su responsabilidad específica.

CuentaPorPagar

Representa la obligación financiera generada por una factura.

Controla monto original, saldo pendiente, fecha de vencimiento y estado financiero.

Cada factura válida genera una cuenta por pagar.

En el diagrama actualizado, la generación de esta cuenta se relaciona con el observador GeneradorCuentaPorPagarAutomatica, que reacciona al registro de una FacturaProveedor y crea la obligación financiera correspondiente.

Además, CuentaPorPagar incluye la operación marcarComoPagadaParcialmente(), utilizada cuando un pago reduce el saldo pendiente pero no cancela completamente la obligación. Esto permite diferenciar explícitamente el estado pagada_parcial del estado pagada.

PagoProveedor

Representa cada pago realizado al proveedor.

Permite registrar fecha, monto, forma de pago y referencia bancaria.

Una cuenta por pagar puede tener múltiples pagos parciales.

Cada pago actualiza automáticamente el saldo pendiente. Si el pago cancela totalmente la obligación, la cuenta se marca como pagada; si aún queda saldo pendiente, se marca como pagada_parcial.

Relaciones principales del modelo

El modelo utiliza relaciones UML para representar cómo interactúan las entidades.

Relación Proveedor — OrdenCompra

Un proveedor puede tener múltiples órdenes de compra, pero cada orden pertenece a un único proveedor.

Relación Proveedor — FacturaProveedor

Un proveedor puede tener múltiples facturas registradas, pero cada factura pertenece a un único proveedor.

Relación Proveedor — CuentaPorPagar

Un proveedor puede tener varias cuentas por pagar asociadas, ya que puede emitir varias facturas durante el ciclo de compras.

Relación OrdenCompra — DetalleOrdenCompra

Es una composición uno a muchos.

Los detalles no existen sin la orden.

Relación OrdenCompra — Recepcion

Una orden puede tener varias recepciones debido a entregas parciales.

Relación Recepcion — DetalleRecepcion

También es una composición.

Cada detalle depende de una recepción específica.

Relación DetalleRecepcion — DetalleOrdenCompra

Cada detalle de recepción se vincula con un detalle de orden de compra para validar que la cantidad recibida no supere la cantidad solicitada.

Relación OrdenCompra — FacturaProveedor

Una orden de compra puede estar asociada a una o varias facturas de proveedor, dependiendo del proceso de compra y facturación.

Relación Recepcion — FacturaProveedor

Una recepción puede estar vinculada a una factura de proveedor cuando la factura corresponde a mercadería efectivamente recibida.

Relación FacturaProveedor — CuentaPorPagar

Cada factura válida genera exactamente una cuenta por pagar.

Relación CuentaPorPagar — PagoProveedor

Una cuenta puede recibir múltiples pagos parciales o un pago total.

Relación GestorCuentasPorPagar — ServicioConsultaCostosA

El GestorCuentasPorPagar implementa la interfaz ServicioConsultaCostosA para exponer al Módulo A consultas de precio de costo de adquisición. Esta relación representa una interfaz provista por el Módulo C, no una dependencia directa hacia las tablas internas del módulo.

Aplicación del patrón Observer

El patrón Observer se incorpora para automatizar procesos que deben ejecutarse como consecuencia de un evento principal, sin acoplar directamente las clases del dominio con todas las acciones secundarias.

En el diagrama se aplica en dos flujos principales:

Observer en el flujo de recepción de mercadería

El sujeto observado es Recepcion.

El evento que activa la notificación es la recepción registrada y completada.

Los observadores son NotificadorInventarioB y ActualizadorEstadoOrdenCompra.

Cuando se registra una recepción, `Recepcion` ejecuta `notificarObservadores()`. A partir de esta notificación, `NotificadorInventarioB` se encarga de comunicar el ingreso de productos al Módulo B mediante `ServicioInventarioB.notificarMovimiento()`, mientras que `ActualizadorEstadoOrdenCompra` actualiza el estado de la orden de compra asociada.

Este diseño permite que el registro de recepción, la notificación de inventario y el cambio de estado de la orden estén conectados funcionalmente, pero separados estructuralmente.

Observer en el flujo de facturación y retenciones

El sujeto observado es `FacturaProveedor`.

El evento que activa la notificación es la factura registrada en estado `pendiente_retencion`.

Los observadores son `EnviadorRetencionModuloD` y `GeneradorCuentaPorPagarAutomatica`.

Cuando se registra una factura, `FacturaProveedor` ejecuta `notificarObservadores()`. A partir de esta notificación, `EnviadorRetencionModuloD` envía los datos al Módulo D mediante `ServicioImpuestosD` para generar la retención, mientras que `GeneradorCuentaPorPagarAutomatica` crea la cuenta por pagar correspondiente.

Este diseño permite automatizar la integración tributaria y la generación de obligaciones financieras sin sobrecargar la clase `FacturaProveedor`.

Beneficios del patrón Observer en el módulo

La aplicación del patrón Observer aporta los siguientes beneficios al diseño:

Reduce el acoplamiento entre las clases principales y los procesos secundarios.

Permite agregar nuevos observadores sin modificar las clases `Recepcion` o `FacturaProveedor`.

Separa responsabilidades dentro de la capa lógica.

Facilita la automatización de procesos posteriores a eventos importantes.

Mejora la mantenibilidad del módulo.

Refuerza la integración con los módulos B y D sin depender directamente de su implementación interna.

Integración entre módulos

El diseño evita dependencias directas entre módulos.

El Módulo C no accede directamente a tablas internas de Inventarios, tablas tributarias, estructuras internas del Módulo A ni procedimientos internos externos. La integración se realiza mediante contratos definidos.

Toda comunicación se realiza mediante interfaces definidas: `ServicioInventarioB` para la integración con el Módulo B, `ServicioImpuestosD` para la integración con el Módulo D y `ServicioConsultaCostosA` para exponer al Módulo A el precio de costo de adquisición en modo solo lectura.

Esto reduce el acoplamiento y facilita cambios internos en otros módulos sin afectar el Módulo C.

Además, esta decisión cumple el requerimiento de integración mediante interfaces y contratos definidos, ya que el Módulo C no depende de la estructura interna de los otros módulos, sino de servicios definidos formalmente.

Con las nuevas adiciones del patrón Observer, esta integración queda mejor organizada. Los servicios externos siguen existiendo como interfaces, pero ahora son utilizados por observadores concretos que reaccionan a eventos específicos del dominio.

Por ejemplo:

NotificadorInventarioB usa ServicioInventarioB para notificar el ingreso de mercadería al Módulo B.

EnviadorRetencionModuloD usa ServicioImpuestosD para enviar la factura al Módulo D.

GeneradorCuentaPorPagarAutomatica crea la CuentaPorPagar cuando se registra una factura.

ActualizadorEstadoOrdenCompra actualiza el estado de la OrdenCompra después de una recepción.

GestorCuentasPorPagar implementa ServicioConsultaCostosA para exponer precios de costo al Módulo A.

De esta forma, el diagrama no solo representa las entidades principales del Módulo C, sino también la forma en que el sistema automatiza sus procesos críticos mediante eventos, observadores e interfaces de integración.

Organización visual del diagrama actualizado

El diagrama actualizado mantiene la separación por capas, pero reorganiza visualmente las clases para facilitar su lectura.

La capa de presentación se mantiene separada y conectada únicamente con el GestorCuentasPorPagar.

La capa de lógica de negocio agrupa el gestor principal, las interfaces de integración y las clases relacionadas con el patrón Observer.

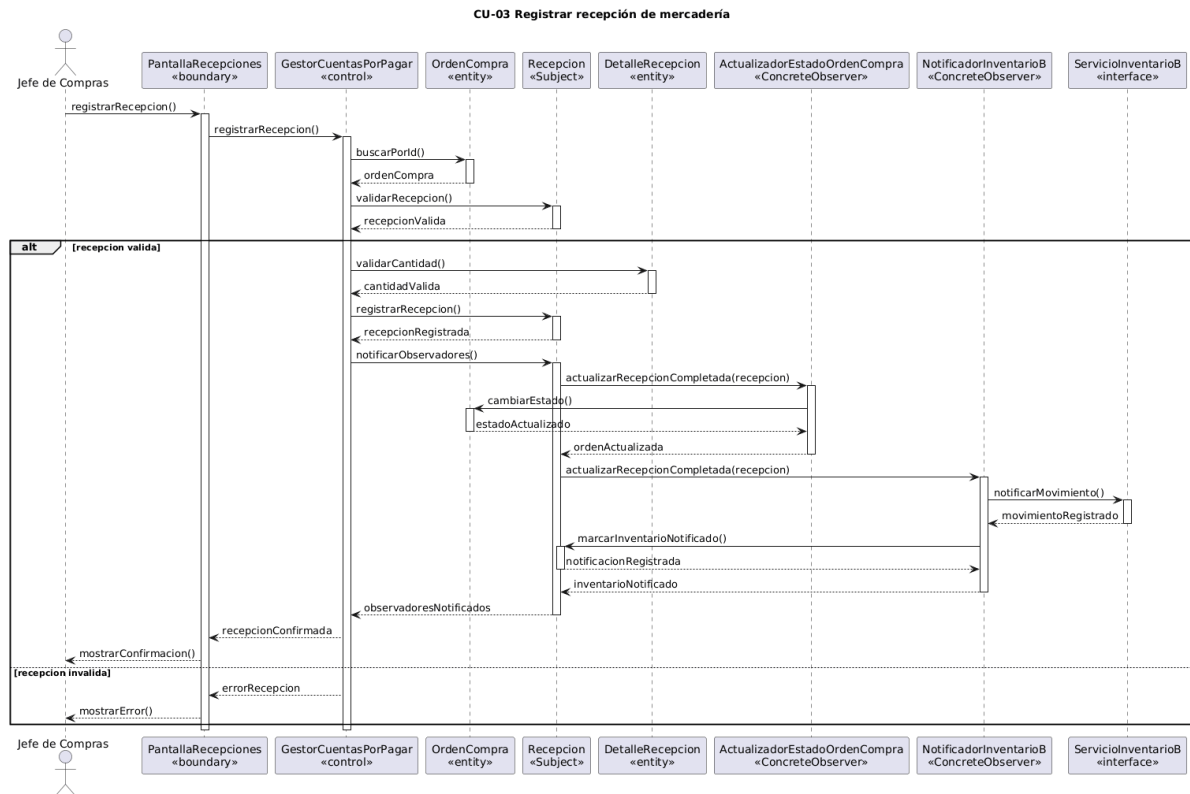
La capa de datos o dominio conserva las entidades principales del módulo, organizadas alrededor del flujo de compra y pago: Proveedor, OrdenCompra, Recepcion, FacturaProveedor, CuentaPorPagar y PagoProveedor.

Las clases DetalleOrdenCompra y DetalleRecepcion complementan el control de productos solicitados y recibidos.

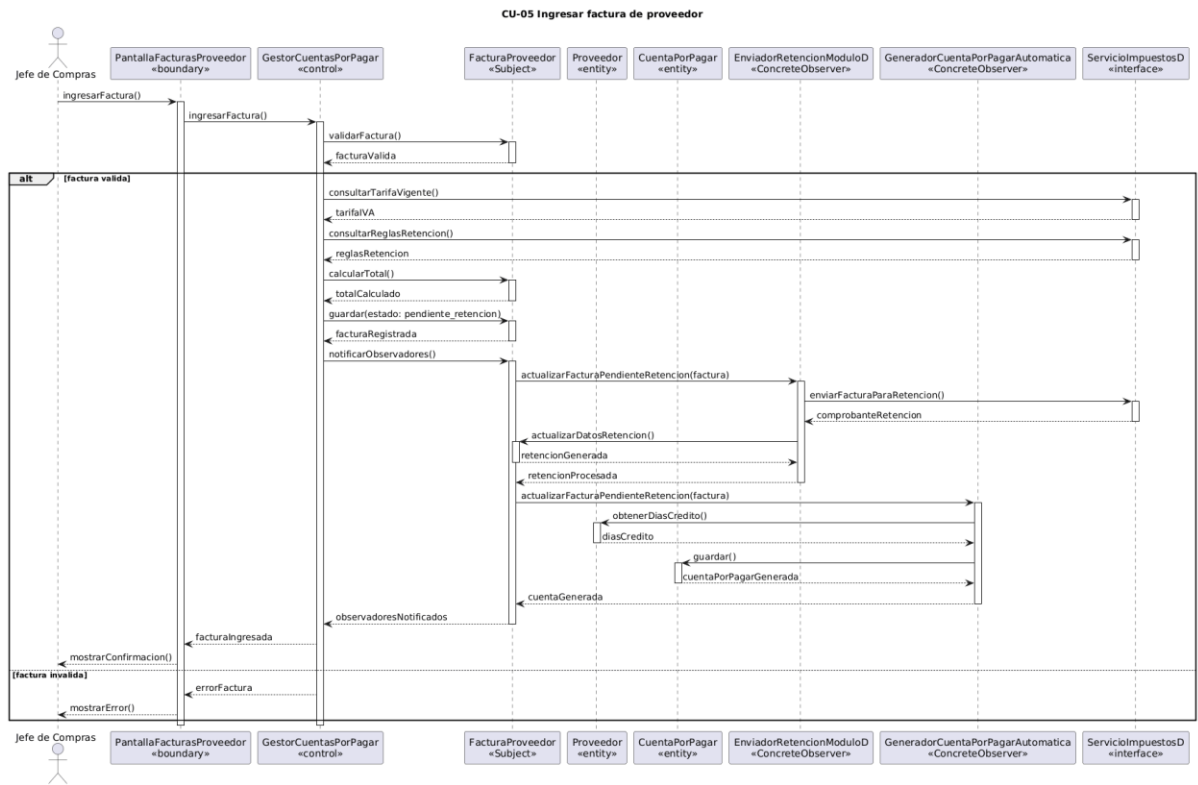
La reorganización no cambia el significado del modelo, sino que mejora la distribución visual de las clases y sus relaciones. El objetivo es que las conexiones sean más claras, que las flechas se crucen menos y que el patrón Observer pueda identificarse con mayor facilidad dentro del diagrama.

e) Modelo comportamental: Diagramas de secuencia

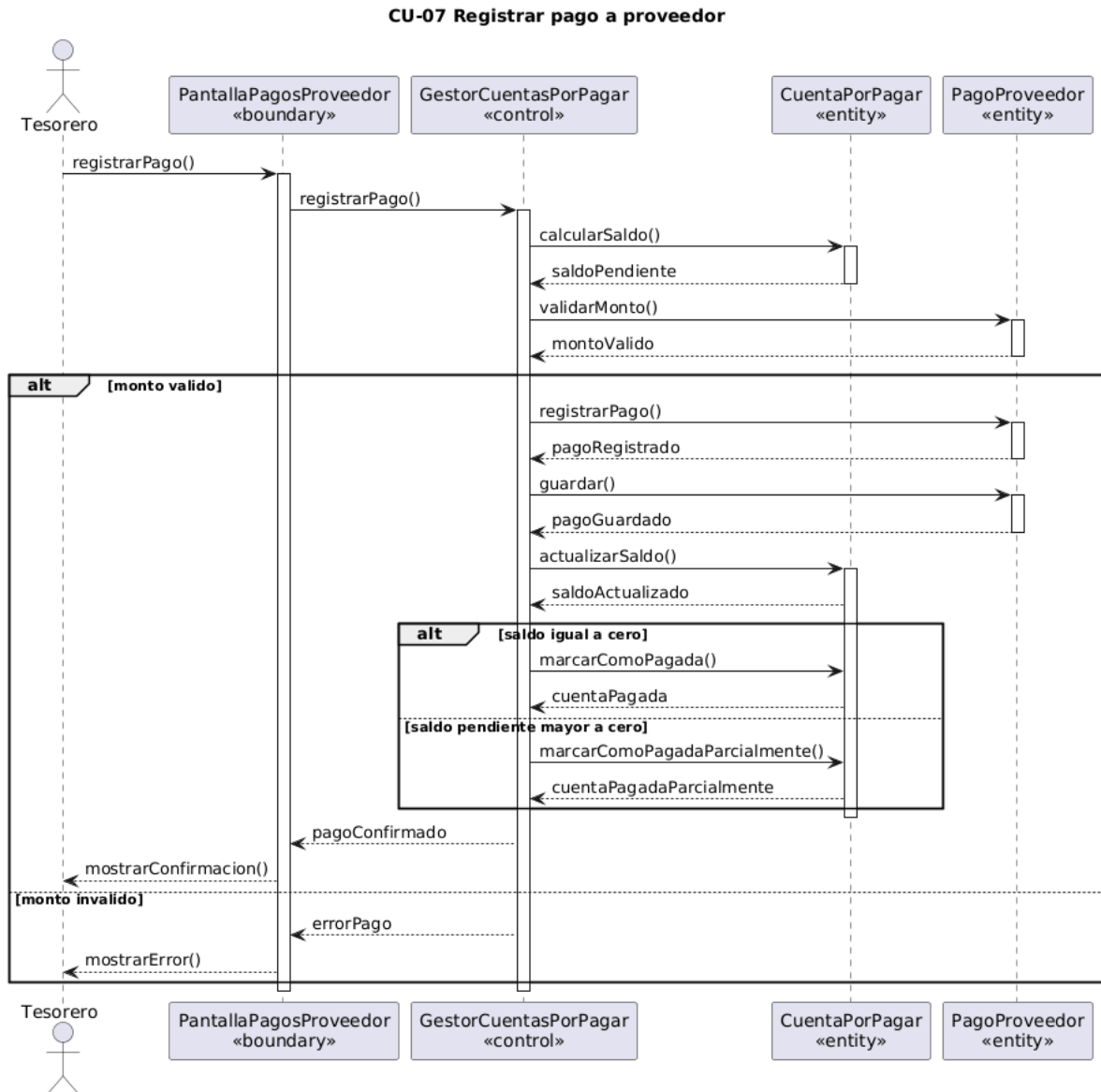
- Mínimo 3 diagramas que representen escenarios principales, elaborados en StarUML.
- Los mensajes deben ser coherentes con las operaciones del diagrama de clases



Link: <https://acortar.link/rQmKSE>



Link: <https://acortar.link/2PoddX>

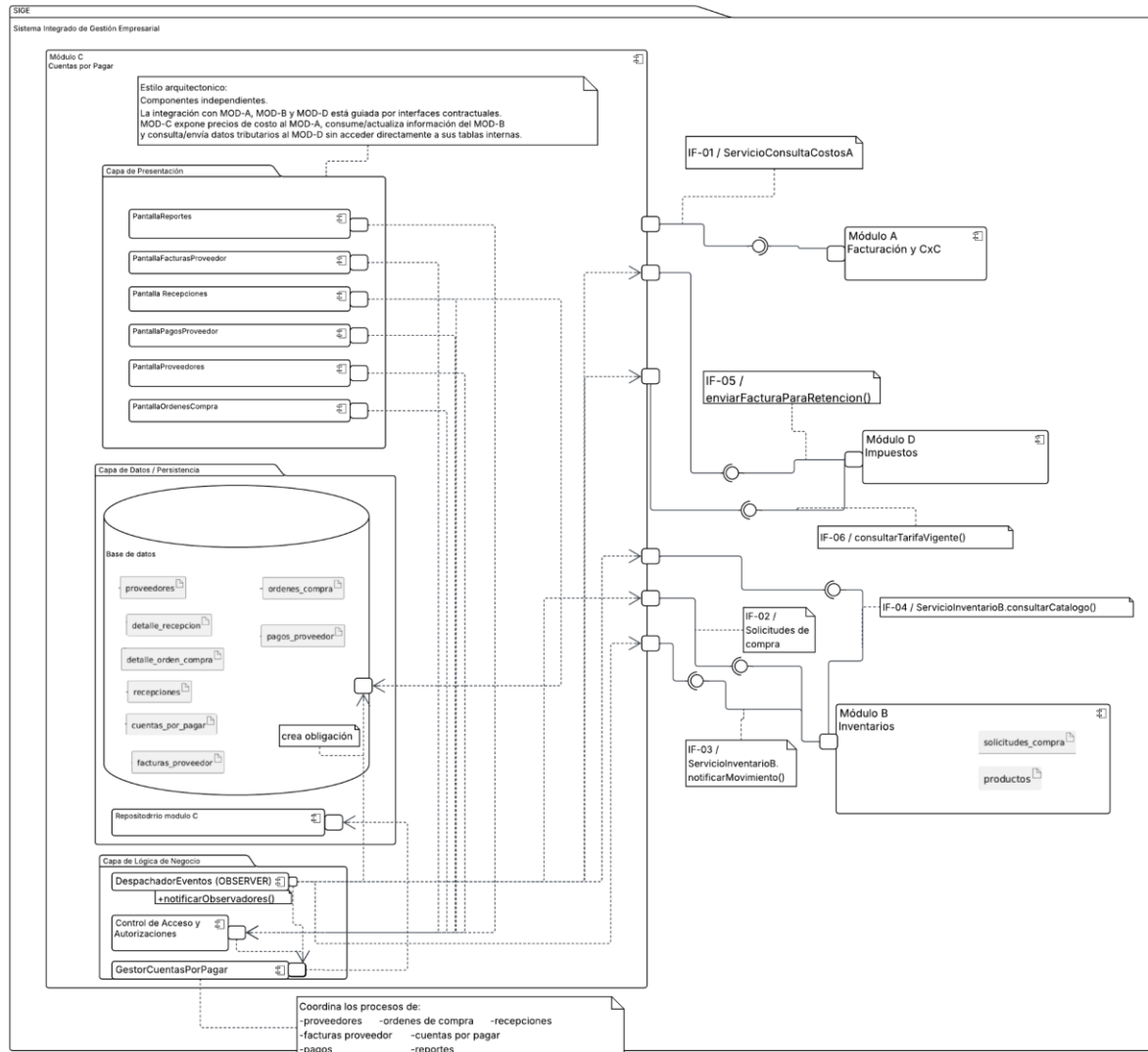


Link: <https://acortar.link/APqhn2>

f) Diseño arquitectónico: Diagrama de componentes

- Posición del módulo dentro de la arquitectura global del SIGE.
- Dependencias e interfaces con los otros módulos. Estilo arquitectónico justificado.

Link: https://lucid.app/lucidchart/38f4484d-aabc-4b9b-a53c-b98c2a549738/edit?viewport_loc=-4895%2C-7918%2C5725%2C3247%2C0_0&invitationId=inv_89b0d5a0-d599-4a47-8336-8242eb749319



ESTILO USADO: Componentes independientes

Justificación

El Módulo C — Cuentas por Pagar adopta el estilo de componentes independientes porque su naturaleza funcional exige que pueda integrarse con los demás módulos del SGE sin generar acoplamiento directo entre sus estructuras internas. Este estilo permite que el módulo mantenga su propia capa de presentación, lógica de negocio y persistencia, mientras se comunica con los demás módulos mediante interfaces contractuales claramente definidas.

A continuación, se argumenta esta decisión desde tres ángulos: la posición del módulo dentro del SGE, las dependencias externas y la relación con los requerimientos no funcionales.

Posición dentro del SGE

El SIGE está compuesto por cuatro módulos: Facturación y CxC (A), Inventarios (B), Cuentas por Pagar (C) e Impuestos (D), los cuales operan de forma autónoma sobre sus propios dominios de datos.

El Módulo C ocupa una posición central en el flujo de compras, ya que recibe solicitudes del Módulo B cuando el stock baja del mínimo, consulta el catálogo de productos del Módulo B para crear órdenes de compra, notifica al Módulo B el ingreso de mercadería al registrar una recepción, consulta tarifas y reglas tributarias al Módulo D al registrar facturas, envía facturas de proveedor al Módulo D para generar retenciones, y expone precios de costo al Módulo A como referencia para el cálculo de márgenes de venta.

Sin embargo, ninguna de estas interacciones requiere acceso directo a las tablas internas de otros módulos. La comunicación se realiza mediante interfaces contractuales: IF-01 para la consulta de precio de costo por parte del Módulo A, IF-02, IF-03 y IF-04 para la integración con el Módulo B, y IF-05 e IF-06 para la integración con el Módulo D.

De esta forma, el Módulo C puede integrarse con el resto del SIGE sin depender directamente de la implementación interna de los demás módulos.

Dependencias mediante interfaces, no acoplamiento directo

La decisión de implementar ServicioInventarioB, ServicioImpuestosD y ServicioConsultaCostosA como interfaces responde al objetivo de reducir el acoplamiento entre módulos.

ServicioInventarioB permite al Módulo C consultar productos, procesar solicitudes de compra y notificar movimientos de inventario sin acceder directamente a la estructura interna del Módulo B. Si el Módulo B cambia la forma en que administra productos, solicitudes o stock, el Módulo C no requiere modificaciones siempre que se respete el contrato definido por las interfaces.

ServicioImpuestosD permite al Módulo C consultar tarifas de IVA, consultar reglas de retención y enviar datos de facturas de proveedor para que el Módulo D genere el comprobante de retención correspondiente. El Módulo C no necesita conocer cómo el Módulo D calcula internamente los impuestos o genera los documentos tributarios; únicamente consume el contrato definido.

Por su parte, ServicioConsultaCostosA representa una interfaz provista por el Módulo C hacia el Módulo A. Mediante IF-01, el Módulo A puede consultar el precio de costo de adquisición en modo solo lectura, sin acceder directamente a las tablas internas del Módulo C.

Las referencias cruzadas entre esquemas de base de datos se representan como campos lógicos, como ref_mod_b_id_producto o ref_mod_d_tarifa_id, evitando llaves foráneas físicas entre esquemas. Esto reduce el acoplamiento a nivel de persistencia y permite que cada módulo conserve autonomía sobre sus propios datos.

Justificación frente a alternativas

Se descartó una arquitectura monolítica clásica porque implicaría que los módulos compartan directamente estructuras internas, tablas o clases, generando mayor dependencia entre ellos. Por ejemplo, un cambio en la estructura de productos del Módulo B podría obligar a modificar el código del Módulo C.

También se descartó una arquitectura de microservicios completa porque el sistema corresponde a una aplicación de gestión empresarial para una PYME, donde la complejidad operacional de servicios distribuidos no se justifica por el volumen ni por la escala del proyecto académico.

El estilo de componentes independientes ofrece un punto medio adecuado: cada módulo conserva su propia responsabilidad funcional, sus propias capas internas y su persistencia, pero se comunica con los demás únicamente a través de interfaces definidas. Esto permite desarrollar, probar y modificar cada módulo de forma separada sin romper la integración global del SIGE.

Relación con los requerimientos no funcionales

Este estilo arquitectónico responde a los requerimientos no funcionales del Módulo C.

RNF01 se cumple porque cada módulo gestiona su propia persistencia y evita consultas directas complejas entre esquemas externos. Esto favorece que las consultas internas del Módulo C, como estados de cuenta y reportes de obligaciones, mantengan tiempos de respuesta adecuados.

RNF02 se cumple porque las transacciones principales del Módulo C son locales a su propio dominio. En caso de fallos de comunicación con los módulos externos, el sistema puede registrar el error en un log de sincronización sin perder los datos ya confirmados dentro del Módulo C.

RNF03 se cumple porque el Módulo C implementa su propio control de acceso y autorizaciones. Las solicitudes provenientes de la capa de presentación pasan por el componente de Control de Acceso y Autorizaciones antes de llegar al GestorCuentasPorPagar. Esto permite distinguir los permisos del Jefe de Compras, del Tesorero y del Administrador del sistema, incluyendo la autorización excepcional de facturas sin orden de compra asociada.

RNF04 se cumple porque la independencia del componente permite implementar una bitácora de auditoría propia sobre las operaciones financieras del módulo, como ingreso de facturas, registro de pagos y actualización de saldos.

RNF05 se cumple directamente porque el uso de ServicioInventarioB, ServicioImpuestosD y ServicioConsultaCostosA como contratos de interfaz garantiza que un cambio interno en los módulos A, B o D no requiera modificar el Módulo C, siempre que se mantengan los contratos definidos.

g) Aplicación de patrones de diseño

- Al menos un patrón identificado, justificado técnicamente y representado en StarUML.

- Patrones sugeridos (no excluyentes): Singleton, Factory Method, Adapter, Observer.

Aplicación del patrón Observer en el Módulo C - Cuentas por Pagar

Beneficios del patrón en el Módulo C

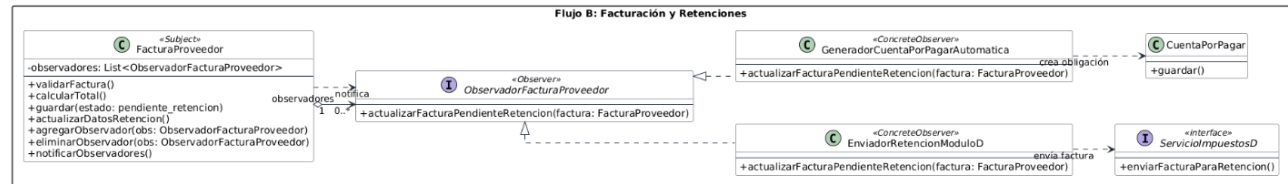
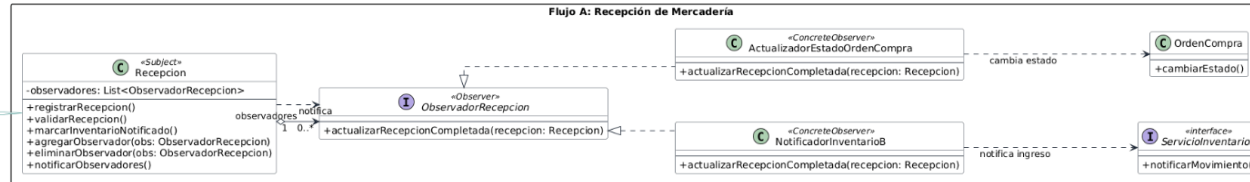
- Bajo acoplamiento entre Subject y Observer.
- Automatización de procesos posteriores.
- Nuevos observadores pueden agregarse sin modificar el Subject.
- Mejor separación de responsabilidades.
- Integración más limpia con Módulo B y Módulo D.

Propósito del patrón Observer

Define una dependencia 1 a N entre objetos. Cuando el Subject cambia de estado, todos sus observadores son notificados automáticamente.

En el Módulo C se aplica para automatizar:

- La recepción de mercadería.
- El registro de facturas de proveedor.
- El envío de datos para retención.
- La generación automática de cuentas por pagar.



Link:

<https://acortar.link/QeZapJ>

h) Matriz de trazabilidad

- Tabla que mapee: Requerimiento → Caso de uso → Clase(s) → Diagrama de secuencia.

Requerimiento	Caso(s) de uso relacionado(s)	Clase(s) / interfaz(es) relacionadas	Diagrama de secuencia asociado
RF01 — Registrar proveedores	CU-01: Gestionar proveedores	PantallaProveedores, GestorCuentasPorPagar, Proveedor, ProveedorPersonaNatural, ProveedorPersonaJuridica	No tiene diagrama de secuencia específico; queda trazado mediante caso de uso y clases.
RF02 — Crear órdenes de compra	CU-02: Crear orden de compra / CU-13: Consultar catálogo	PantallaOrdenesCompra, GestorCuentasPorPagar, OrdenCompra, DetalleOrdenCompra, Proveedor, ServicioInventarioB	No tiene diagrama de secuencia específico; queda trazado mediante caso de uso, clases e interfaz con Módulo B.
RF03 — Registrar recepciones parciales	CU-03: Registrar recepción de mercadería	PantallaRecepciones, GestorCuentasPorPagar, OrdenCompra, Recepcion, DetalleRecepcion, ObservadorRecepcion, ActualizadorEstadoOrdenCompra	DS-01: CU-03 Registrar recepción de mercadería
RF04 — Actualizar inventario Módulo B	CU-04: Actualizar inventario	Recepcion, DetalleRecepcion, ObservadorRecepcion, NotificadorInventarioB, ServicioInventarioB	DS-01: CU-03 Registrar recepción de mercadería, porque CU-04 se ejecuta como comportamiento incluido desde CU-03.

RF05 — Ingresar facturas proveedor	CU-05: Ingresar factura de proveedor / CU-15: Consultar tarifa IVA y retenciones / CU-16: Enviar datos factura a Módulo D	PantallaFacturasProveedor, GestorCuentasPorPagar, FacturaProveedor, Proveedor, OrdenCompra, Recepcion, ServicioImpuestosD, EnviadorRetencionModuloD	DS-02: CU-05 Ingresar factura de proveedor
RF06 — Generar cuenta por pagar	CU-06: Generar cuenta por pagar	FacturaProveedor, Proveedor, CuentaPorPagar, ObservadorFacturaProveedor, GeneradorCuentaPorPagarAutomatica	DS-02: CU-05 Ingresar factura de proveedor, porque CU-06 se ejecuta como comportamiento incluido desde CU-05.
RF07 — Registrar pagos	CU-07: Registrar pago a proveedor	PantallaPagosProveedor, GestorCuentasPorPagar, CuentaPorPagar, PagoProveedor, Proveedor	DS-03: CU-07 Registrar pago a proveedor
RF08 — Actualizar saldo pendiente	CU-08: Actualizar saldo pendiente	GestorCuentasPorPagar, CuentaPorPagar, PagoProveedor	DS-03: CU-07 Registrar pago a proveedor, porque CU-08 se ejecuta como comportamiento incluido desde CU-07.
RF09 — Estado de cuenta por proveedor	CU-09: Generar estado de cuenta	PantallaReportes, GestorCuentasPorPagar, Proveedor, FacturaProveedor, CuentaPorPagar, PagoProveedor	No tiene diagrama de secuencia específico; queda trazado mediante caso de uso y clases.

RF10 — Alertar pagos por vencer	CU-10: Alertar pagos próximos a vencer	PantallaReportes, GestorCuentasPorPagar, CuentaPorPagar, FacturaProveedor, Proveedor	No tiene diagrama de secuencia específico; se relaciona como extensión de CU-09 Generar estado de cuenta.
RF11 — Consultar OC pendientes	CU-11: Consultar órdenes pendientes	PantallaOrdenesCompra, GestorCuentasPorPagar, OrdenCompra, DetalleOrdenCompra, Recepcion, DetalleRecepcion	No tiene diagrama de secuencia específico; queda trazado mediante caso de uso y clases.
RF12 — Reporte obligaciones pendientes	CU-12: Generar reporte de obligaciones	PantallaReportes, GestorCuentasPorPagar, Proveedor, FacturaProveedor, CuentaPorPagar, PagoProveedor	No tiene diagrama de secuencia específico; queda trazado mediante caso de uso y clases.
RF13 — Notificar ingreso inventario	CU-03: Registrar recepción / CU-04: Actualizar inventario	Recepcion, DetalleRecepcion, ObservadorRecepcion, NotificadorInventarioB, ServicioInventarioB	DS-01: CU-03 Registrar recepción de mercadería
RF14 — Consultar catálogo Módulo B	CU-02: Crear orden de compra / CU-13: Consultar catálogo de productos	PantallaOrdenesCompra, GestorCuentasPorPagar, OrdenCompra, DetalleOrdenCompra, ServicioInventarioB	No tiene diagrama de secuencia específico; se ejecuta como comportamiento incluido desde CU-02 Crear orden de compra.

RF15 — Solicitud compra Módulo B	CU-02: Crear orden de compra	PantallaOrdenesCompra, GestorCuentasPorPagar, OrdenCompra, DetalleOrdenCompra, ServicioInventarioB	No tiene diagrama de secuencia específico; queda trazado mediante caso de uso, clases e interfaz con Módulo B.
RF16 — Exponer precio de costo	CU-14: Exponer precio de costo	ServicioConsultaCostosA, GestorCuentasPorPagar, OrdenCompra, DetalleOrdenCompra	No tiene diagrama de secuencia específico; queda trazado mediante caso de uso, clases e interfaz provista al Módulo A.
RF17 — Gestión IVA y retenciones	CU-05: Ingresar factura / CU-15: Consultar tarifa IVA y retenciones / CU-16: Enviar datos factura a Módulo D	PantallaFacturasProveedor, GestorCuentasPorPagar, FacturaProveedor, ServicioImpuestosD, ObservadorFacturaProveedor, EnviadorRetencionModuloD	DS-02: CU-05 Ingresar factura de proveedor
RNF03 — Control de acceso por roles	CU-17: Autorizar factura excepcional sin OC	PantallaFacturasProveedor, GestorCuentasPorPagar, FacturaProveedor	No tiene diagrama de secuencia específico; se relaciona con el flujo alternativo de CU-05 Ingresar factura de proveedor.

i) Conclusiones individuales

- Máximo media página por integrante: aprendizajes, dificultades y mejoras propuestas.

❖ Daniel Maldonado

En lo personal, me permitió aprender más en cuanto a los aspectos del modelado de software que no vimos en clase. Si bien durante el curso se abordaron los diagramas fundamentales de UML como casos de uso, clases y secuencia, la realización del Módulo C me llevó a explorar herramientas y tipos de diagramas que no habían sido parte del contenido formal de la materia en sí.

Si hablamos de dificultades, diría que la redacción técnica de los documentos. Describir diagramas, códigos, y todo lo relacionado. Comunicar ideas técnicas por escrito de manera que sean comprensibles, a diferencia del modelado o el código, se practica poco en el aula y que este proyecto me exigió desarrollar de forma más consciente.

Finalmente, diría que el proyecto me permitió comprender con mayor profundidad el diagrama de componentes, el cual no vimos profundamente en clase; normalmente no se trabaja con el mismo detalle que los diagramas de clases o secuencia. Este tipo de diagrama fue clave para representar las interfaces de comunicación entre el Módulo C y los módulos A, B y D del SIGE, evidenciando cómo los contratos entre componentes permiten que el sistema evolucione de forma independiente sin romper la integración.

❖ Marco Rojas

Aprendizajes: El desarrollo de este proyecto mejoró considerablemente mi comprensión sobre la importancia de la cohesión e interconexión en los sistemas, especialmente en ERP complejos. Me ayudó a entender la utilidad en la implementación de patrones de diseño como soluciones a problemas generales en el desarrollo de productos de software; en nuestro caso, el patrón Observer fue el más indicado debido a la interconexión con el resto de módulos. Reforzar buenas prácticas como que el gestor principal dependa de sus implementaciones concretas conectándose de forma indirecta con el resto de los módulos y logrando una estructura por capas fácil de modificar fueron las lecciones más importantes que me llevo de este proyecto.

Dificultades: La principal estuvo en el diseño de las dependencias con módulos externos. Modelar el cómo se relaciona nuestra clase Observer con el resto de interfaces requirió de varias correcciones en el diseño de los diagramas y flujo de nuestro propio módulo. Coordinar la consistencia e integridad de los datos y sus dominios con el resto de módulos también representó una complejidad.

Mejoras propuestas: Establecer en una etapa más temprana del desarrollo pautas y contratos con el resto de módulos, así como acordar patrones y estilos de diseño desde el inicio para evitar los cambios en exceso dentro del flujo de los módulos.

❖ Andres Merchan

Durante el desarrollo de este proyecto aprendí que trabajar en un sistema grande no solo implica elaborar diagramas o cumplir con entregables, sino también saber organizarse desde el inicio, coordinar tareas con el equipo y mantener una visión general de todo el trabajo. Comprendí que cada parte del proyecto debe construirse con coherencia, ya que los requerimientos, casos de uso, diagramas de clases, diagramas de secuencia, componentes y matriz de trazabilidad están conectados entre sí. También entendí mejor la importancia de UML, porque cada diagrama cumple una función específica y permite representar de forma gráfica y resumida la información necesaria para planificar correctamente el sistema.

Una de las principales dificultades fue realizar diagramas que no había trabajado antes con tanta profundidad y asegurar que todos los artefactos mantuvieran relación entre sí. Además, el trabajo en equipo presentó retos, especialmente al coordinar con diferentes formas de participación y compromiso. Sin embargo, junto con mi equipo logramos sobrellevar esas dificultades mediante organización, comunicación y apoyo entre los integrantes que estuvieron constantemente involucrados en el desarrollo del proyecto.

Como mejora, considero que en futuros proyectos sería importante llevar un mejor control de la participación individual y grupal, para que el esfuerzo de quienes trabajan activamente no se vea afectado por la falta de aporte de otros integrantes. También sería útil organizar desde el primer día el uso de herramientas como GitHub, definir responsabilidades claras y planificar cada sección con anticipación. En general, este proyecto me permitió comprender mejor la importancia de la planificación, la documentación técnica y la coherencia en el diseño de software.

❖ Ariel Serrano

Luego de haber realizado el proyecto he podido llegar a las siguientes conclusiones:

Aprendí que el trabajo con una base de datos del estilo distribuida requiere de comunicación continua por parte de cada uno de los módulos, también aprendí que los contratos y acuerdos de ese estilo son esenciales para los avances y la implementación correcta del código para cada módulo, puesto que los módulos necesitan pedir ciertas cosas a otros módulos y saber qué esperan recibir los otros módulos, con esto se puede asegurar la correcta implementación de todas las funcionalidades.

Al ser la primera vez que se aplica este proyecto, siento que faltó un poco de guía pero se pudo resolver, quizá mostrar ejemplos de este proyecto podría ayudar, la comunicación entre grupos también fue un inconveniente fundamental sobre todo

porque a veces la falta de respuesta de los grupos, o respuesta tardía de los módulos, dificultaba el avance.

Para poder solucionar todos estos problemas yo sugiero que se establezca desde el inicio del proyecto un canal de comunicación en el cual se encuentre el docente, pues con esto se podría generar un seguimiento adecuado del tiempo en que se demoraron en responder los módulos y la colaboración que están ofreciendo, además este mismo puede servir para resolver dudas o, si es que se llegaron a encontrar, incoherencias entre los diferentes módulos, considero que la implementación de los entregables de los avances fue bastante acertado y que se debe seguir haciendo, pues esto asegura que todos los módulos vayan avanzando de manera uniforme y así que no haya módulos que estén atrasados con respecto al avance de los demás.

Declaración de uso de herramientas de IA

Se utilizó ChatGPT como herramienta de apoyo para revisar la redacción técnica, la organización del informe, la coherencia entre artefactos UML y la validación del documento frente a la rúbrica del proyecto. Los resultados fueron revisados y validados por el equipo mediante comparación con los requerimientos del Módulo C, revisión manual de los diagramas y verificación de que las decisiones de diseño mantuvieran consistencia con el documento guía y con los acuerdos de integración entre módulos.

j) Referencias bibliográficas

- Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.
- Larman, C. (2005). Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development (3rd ed.). Prentice Hall.
- Lucid Software Inc. (2025). Lucidchart documentation. <https://www.lucidchart.com/pages/>
- MKLLabs Co., Ltd. (2025). StarUML user guide. <https://docs.staruml.io/>
- Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), Version 2.5.1. <https://www.omg.org/spec/UML/2.5.1/>
- Oracle. (2024). Java Platform, Standard Edition documentation. <https://docs.oracle.com/javase/>
- Oracle Corporation. (2024). MySQL 8.0 reference manual. <https://dev.mysql.com/doc/>
- PlantUML. (2025). PlantUML language reference guide. <https://plantuml.com/>
- Pressman, R. S., & Maxim, B. R. (2020). Ingeniería del software: Un enfoque práctico (9.ª ed.). McGraw-Hill Education.
- Sommerville, I. (2016). Ingeniería de software (10.ª ed.). Pearson Educación.